

Debug Overlay — Hướng dẫn dùng cho mọi dự án web/PWA

Đoạn code nhỏ, hiện thông số debug (kích thước, biến số, trạng thái) ngay trên màn hình bằng chữ xanh lá nền đen — không cần remote debug qua máy tính, dùng được cả trên trình duyệt thường lẫn app PWA đã cài.

Mục lục

1. [4 cách chọn điều kiện bật](#)
2. [Lưu ý khi đổi sang dự án khác](#)
3. [Bổ sung: giá trị sẵn có của trình duyệt](#)
4. [Bổ sung: `visualViewport` — bắt đúng lúc thanh hệ thống ẩn/hiện](#)
5. [Bổ sung: `trigger resume` — khi nào cần gọi lại debug](#)
6. [Code đầy đủ cho cả 4 cách](#)
7. [Cách xem trên trình duyệt thường](#)
8. [Cách xóa hoàn toàn sau khi debug xong](#)
9. [Lưu ý khi tái sử dụng cho dự án khác](#)

4 cách chọn điều kiện bật (`DEBUG_ON`)

Cách	Điều kiện bật	Ưu điểm	Nhược điểm
1. Luôn bật	<code>true</code> cứng	Đơn giản nhất, chắc chắn hiện	Phải nhớ tự xóa/tắt trước khi deploy
2. Qua URL	<code>?debug=1</code>	Bật/tắt linh hoạt, không sửa code	PWA mở từ home screen thường mất

Cách	Điều kiện bật	Ưu điểm	Nhược điểm
			query string
3. Qua <code>localStorage</code>	<code>localStorage.getItem('debug')==='1'</code>	Bật 1 lần dùng mãi, hoạt động cả PWA lẫn browser vì chung storage	Cần 1 lần thao tác Console để bật ban đầu
4. Tự động theo <code>display-mode</code>	<code>matchMedia('(display-mode: standalone)').matches</code>	Không cần làm gì cả — cứ mở PWA là tự hiện ngay	Rủi ro cao nhất: nếu quên xóa, MỌI người dùng PWA thật đều thấy debug info

Khuyến nghị: dùng Cách 4 khi đang debug 1 bug cụ thể liên quan đến PWA/standalone (như tình huống Gem Grid), vì tiện nhất — nhưng **bắt buộc phải xóa ngay sau khi xong việc**, không được để sót khi deploy bản chính thức cho người dùng.

 Lưu ý quan trọng khi đổi sang dự án khác

Cả 4 code mẫu bên dưới đều đã điền sẵn 5 thông số dùng để tìm bug Gem Grid (`cellPx`, `board`, `tray`, `trayCellPx`, `display-mode`) làm ví dụ đây

đủ. Nhưng `boardEl`, `trayEl`, `cellPx`, `trayCellPx()` là **tên biến/hàm riêng của dự án Gem Grid** — dự án khác sẽ không có sẵn, phải tự đổi lại.

Quy tắc chung: muốn hiện biến gì, viết đúng tên biến đó

- 1. Mở file dự án của bạn, tìm tên biến/hàm liên quan đến vấn đề đang debug (ví dụ: biến lưu kích thước, phần tử DOM đang nghi bị lệch)
- 2. Trong `updateDbg()`, nối thêm `' | tênBiến='+giá_trị` vào chuỗi `dbg.textContent`, dùng đúng tên biến thật lấy từ bước 1
- 3. Nếu biến là 1 phần tử DOM (ví dụ `document.getElementById('abc')`), cần gọi `.getBoundingClientRect()` để lấy kích thước thực tế, như ví dụ `boardEl` / `trayEl` bên dưới
- 4. Nếu biến là số thường (ví dụ `cellPx`), chỉ cần `.toFixed(1)` để làm tròn cho dễ đọc, không cần `getBoundingClientRect()`

Bổ sung: Các giá trị sẵn có của trình duyệt (không phụ thuộc dự án)

Những giá trị dưới đây **không cần tìm trong code dự án** — dán vào trang web nào cũng dùng được, đặc biệt hữu ích khi debug tỷ lệ hiển thị, responsive, hoặc vấn đề PWA.

Bảng giá trị sẵn có

Muốn đo	Code	Ghi chú
Kích thước viewport	<code>window.innerWidth / window.innerHeight</code>	Thay đổi khi resize hoặc xoay máy
Kích thước toàn trang	<code>document.body.scrollHeight / scrollWidth</code>	Kể cả phần bị ẩn ngoài màn hình
Vị trí đã cuộn	<code>window.scrollX / window.scrollY</code>	
Tỉ lệ pixel thiết bị	<code>window.devicePixelRatio</code>	Màn Retina/HiDPI = 2 hoặc 3, màn thường = 1
Hướng màn hình	<code>screen.orientation.type</code>	<code>portrait-primary</code> hoặc <code>landscape-primary</code>

Muốn đo	Code	Ghi chú
Kích thước màn hình vật lý	<code>screen.width / screen.height</code>	Không đổi khi resize, khác với viewport
Chế độ hiển thị	<code>window.matchMedia('(display-mode: standalone)').matches</code>	Đã có trong tài liệu gốc

Ví dụ dùng ngay — không cần biết dự án dùng biến gì

Đây là đoạn debug overlay hoàn chỉnh **chỉ dùng giá trị có sẵn của trình duyệt**, không phụ thuộc bất kỳ biến nào của dự án. Thích hợp nhất cho mục đích debug tỷ lệ hiển thị, responsive, PWA:

```
/* === DEBUG OVERLAY — XÓA TRƯỚC KHI DEPLOY === */
if(true){
  var dbg = document.createElement('div');
  dbg.style.cssText = 'position:fixed;top:0;left:0;right:0;z-index:99999;'+
    'background:rgba(0,0,0,.85);color:#0f0;font:11px monospace;'+
    'padding:6px 8px;white-space:pre-wrap:pointer-events:none;';
  document.body.appendChild(dbg);
  function updateDbg(){
    dbg.textContent =
      'viewport=' + window.innerWidth + 'x' + window.innerHeight +
      ' | dpr=' + window.devicePixelRatio +
      ' | orientation=' + screen.orientation.type +
      ' | display-mode=' + (window.matchMedia('(display-mode: standalone)').matches ? 'standalone' : 'browser');
  }
  window.addEventListener('resize', updateDbg); // cập nhật ngay khi resize/xoay máy
  setInterval(updateDbg, 300);
  updateDbg();
}
/* === HẾT KHỐI DEBUG OVERLAY === */
```

Khi nào cần thêm biến của dự án?

- **Chỉ dùng giá trị trình duyệt** là đủ khi debug: tỷ lệ hiển thị, responsive, PWA standalone, xoay màn hình
- **Cần thêm biến của dự án** khi debug logic nội bộ — ví dụ kích thước 1 ô trong game (`cellPx`), font tự tính, padding động, layout do JS

tính toán

Ví dụ kết hợp cả hai:

```
function updateDbg(){
  // Giá trị có sẵn của trình duyệt
  var info = 'viewport=' + window.innerWidth + 'x' + window.innerHeight
+
  ' | dpr=' + window.devicePixelRatio;
  // Biến riêng của dự án – chỉ thêm khi cần debug logic nội bộ
  var card = document.getElementById('card');
  if(card){
    var r = card.getBoundingClientRect();
    info += ' | card=' + r.width.toFixed(0) + 'x' + r.height.toFixed(0);
  }
  dbg.textContent = info;
}
```

Bổ sung: `visualViewport` — bắt đúng lúc thanh hệ thống ẩn/hiện

`window.innerHeight` và `100dvh / 100svh` đo kích thước viewport, nhưng **không tự báo khi giá trị đó thay đổi** ngoài sự kiện `resize` thông thường (đổi cỡ cửa sổ, xoay máy). Trên PWA standalone, khi status bar/navigation bar của hệ thống ẩn hoặc hiện lại (ví dụ vuốt từ cạnh màn hình, hoặc lúc app vừa mở còn đang animate vào full-immersive), viewport thực tế đổi nhưng **không phải lúc nào cũng bắn ra sự kiện `resize` chuẩn**.

`window.visualViewport` là API chuyên để bắt đúng những thay đổi này — đây chính là API đã dùng để xác nhận và sửa bug tray-bé trong dự án Gem Grid.

```
if(window.visualViewport){
  window.visualViewport.addEventListener('resize', function(){
    console.log('visualViewport đổi:',
      window.visualViewport.width, 'x', window.visualViewport.height);
    updateDbg(); // gọi lại debug overlay ngay khi viewport thật đổi
  });
}
```

Thêm vào bảng giá trị sẵn có:

Muốn đo	Code	Ghi chú
Kích thước	<code>window.visualViewport.width / .height</code>	Chính xác hơn <code>innerWidth / inner</code>

Muốn đo	Code	Ghi chú
viewport thật (trừ bàn phím ảo, thanh hệ thống)		trên mobile khi có bài hoặc thanh hệ thống
Sự kiện khi viewport thật đổi	<code>window.visualViewport.addEventListener('resize', fn)</code>	Bắt được cả trường h <code>resize</code> thường khô ra

Khi nào dùng: nghi ngờ layout bị lệch/cắt trên mobile, đặc biệt trong PWA standalone hoặc khi có bàn phím ảo hiện lên — `innerHeight` thường "trễ" theo, còn `visualViewport` phản ánh đúng ngay lập tức.

Bổ sung: Trigger resume — khi nào cần gọi lại debug (ngoài `setTimeout`)

Một loại bug đặc biệt: **giá trị sai lúc mở app lần đầu, nhưng tự đúng lại nếu thoát ra Home rồi mở lại** (app vẫn sống trong Recent Apps, không bị kill hẳn). Đây là dấu hiệu của **resume-từ-background**, không phải lỗi giá trị tính toán — dùng `setTimeout` đoán mò thời gian sẽ không giải quyết được tận gốc, vì vấn đề không nằm ở "chưa đủ thời gian" mà ở "trang chưa từng được yêu cầu tính lại layout".

3 sự kiện nên gán cùng lúc để bắt đúng thời điểm này:

```
// Bắt lúc tab/app chuyển từ ẩn sang hiện (thoát Home rồi mở lại)
document.addEventListener('visibilitychange', function(){
  if(document.visibilityState === 'visible') updateDbg();
});

// Bắt lúc trình duyệt phục hồi trang từ bfcache (back/forward cache)
window.addEventListener('pageshow', function(){ updateDbg(); });

// Dự phòng thêm cho một số thiết bị/trình duyệt không bắn 2 sự kiện trên
window.addEventListener('focus', function(){ updateDbg(); });
```

Cách nhận diện đây có phải đúng loại bug này không: thử thao tác thoát ra màn hình Home (không tắt hẳn app, không xóa khỏi Recent Apps), rồi mở lại. Nếu giá trị/layout tự đúng ngay lập tức — đây chính là bug resume, cần 3 listener trên thay vì cố tăng thời gian `setTimeout`.

Code đầy đủ cho cả 4 cách

Cấu trúc chung giống nhau, chỉ khác dòng điều kiện `if(...)` đầu tiên. Phần thân bên trong (tạo `div`, `updateDbg()`, `setInterval`) giữ nguyên cho cả 4 cách — chỉ đổi điều kiện bật.

Cách 1 — Luôn bật cứng

Dùng khi đang code/test tại chỗ, chưa cần bật/tắt linh hoạt.

```
/* == DEBUG OVERLAY (Cách 1: luôn bật) — XÓA TRƯỚC KHI DEPLOY == */
if(true){
  var dbg=document.createElement('div');
  dbg.style.cssText='position:fixed;top:0;left:0;right:0;z-index:99999;'+
    'background:rgba(0,0,0,.85);color:#0f0;font:11px monospace;'+
    'padding:6px 8px;white-space:pre-wrap:pointer-events:none;';
  document.body.appendChild(dbg);
  function updateDbg(){
    /* ⚠ Đây là tên biến của Gem Grid — đổi lại đúng tên biến thật
      trong dự án của bạn (xem mục "Quy tắc chung" bên dưới) */
    var br=boardEl.getBoundingClientRect();
    var tr=trayEl.getBoundingClientRect();
    dbg.textContent=
      'cellPx='+cellPx.toFixed(1)+
      ' | board='+br.width.toFixed(0)+'x'+br.height.toFixed(0)+
      ' | tray='+tr.width.toFixed(0)+'x'+tr.height.toFixed(0)+
      ' | trayCellPx='+trayCellPx().toFixed(1)+
      ' | display-mode='+window.matchMedia('(display-mode: standalone)').matches?'standalone':'browser';
  }
  setInterval(updateDbg,300);
  updateDbg();
}
/* == HẾT KHỐI DEBUG OVERLAY == */
```

Cách 2 — Bật qua query string trên URL

Dùng khi muốn bật/tắt mà không sửa code, và đang test trên trình duyệt thường (không phải PWA đã cài — vì PWA mất query string khi mở từ home screen).

```

/* == DEBUG OVERLAY (Cách 2: qua ?debug=1) – XÓA TRƯỚC KHI DEPLOY == */
if(location.search.indexOf('debug=1')!=-1){
  var dbg=document.createElement('div');
  dbg.style.cssText='position:fixed;top:0;left:0;right:0;z-
index:99999;'+
  'background:rgba(0,0,0,.85);color:#0f0;font:11px monospace;'+
  'padding:6px 8px;white-space:pre-wrap:pointer-events:none;';
  document.body.appendChild(dbg);
  function updateDbg(){
    /* ⚠️ Tên biến của Gem Grid – đổi lại theo dự án của bạn */
    var br=boardEl.getBoundingClientRect();
    var tr=trayEl.getBoundingClientRect();
    dbg.textContent=
      'cellPx='+cellPx.toFixed(1)+
      ' | board='+br.width.toFixed(0)+'x'+br.height.toFixed(0)+
      ' | tray='+tr.width.toFixed(0)+'x'+tr.height.toFixed(0)+
      ' | trayCellPx='+trayCellPx().toFixed(1)+
      ' | display-mode='+window.matchMedia('(display-mode:
standalone)').matches?'standalone':'browser');
  }
  setInterval(updateDbg,300);
  updateDbg();
}
/* == HẾT KHỐI DEBUG OVERLAY == */

```

Cách bật: vào `trang.html?debug=1` trên trình duyệt.

Cách 3 – Bật qua `localStorage` (khuyến dùng để test PWA)

Bật 1 lần, dùng lại được mãi mãi ở cả trình duyệt lẫn PWA, vì

`localStorage` dùng chung theo origin bất kể mở qua đường nào.

```

/* == DEBUG OVERLAY (Cách 3: qua localStorage) – XÓA TRƯỚC KHI DEPLOY
== */
if(localStorage.getItem('debug')==='1'){
  var dbg=document.createElement('div');
  dbg.style.cssText='position:fixed;top:0;left:0;right:0;z-
index:99999;'+
  'background:rgba(0,0,0,.85);color:#0f0;font:11px monospace;'+
  'padding:6px 8px;white-space:pre-wrap:pointer-events:none;';
  document.body.appendChild(dbg);
  function updateDbg(){
    /* ⚠️ Tên biến của Gem Grid – đổi lại theo dự án của bạn */
    var br=boardEl.getBoundingClientRect();
    var tr=trayEl.getBoundingClientRect();
    dbg.textContent=
      'cellPx='+cellPx.toFixed(1)+
      ' | board='+br.width.toFixed(0)+'x'+br.height.toFixed(0)+
      ' | tray='+tr.width.toFixed(0)+'x'+tr.height.toFixed(0)+

```



```

    ' | trayCellPx='+trayCellPx().toFixed(1)+
    ' | display-mode='+window.matchMedia('(display-mode:
standalone)').matches?'standalone':'browser');
  }
  setInterval(updateDbg,300);
  updateDbg();
}
/* == HẾT KHỐI DEBUG OVERLAY == */

```

Cách bật (làm 1 lần trước khi cài PWA):

1. Mở trang qua trình duyệt thường (chưa cài PWA)
2. Bấm F12 → tab Console
3. Gõ: `localStorage.setItem('debug', '1')` → Enter
4. Tải lại trang — debug overlay hiện ngay
5. Từ giờ, dù mở qua trình duyệt hay cài PWA mới, debug overlay vẫn tự hiện vì đã lưu trong `localStorage` của origin đó

Cách tắt: gõ `localStorage.removeItem('debug')` trong Console rồi tải lại trang.

Cách 4 — Tự động theo `display-mode` (khuyến dùng khi debug bug PWA cụ thể)

Không cần thao tác gì cả — cứ mở app PWA đã cài là tự động hiện ngay lập tức. Đây là cách đã dùng để tìm ra bug tray-bé trong dự án Gem Grid.

Dán đoạn này vào **cuối file**, ngay trước `})();` hoặc trước `</script>` đóng cuối cùng của file:

```

/*
=====
=
DEBUG OVERLAY — XÓA TOÀN BỘ KHỐI NÀY TRƯỚC KHI DEPLOY BẢN CHÍNH THỨC
Tự động hiện khi chạy ở chế độ PWA standalone (đã cài vào máy),
không cần thao tác gì thêm. Hiện chữ xanh lá trên nền đen ở đầu
màn hình, cập nhật liên tục mỗi 300ms.

=====
= */
if(window.matchMedia('(display-mode: standalone)').matches){
  var dbg=document.createElement('div');
  dbg.style.cssText='position:fixed;top:0;left:0;right:0;z-
index:99999;'+
    'background:rgba(0,0,0,.85);color:#0f0;font:11px monospace;'+
    'padding:6px 8px;white-space:pre-wrap:pointer-events:none;';
  document.body.appendChild(dbg);
  function updateDbg(){

```

```

/* ⚠️ Tên biến của Gem Grid – đổi lại theo dự án của bạn */
var br=boardEl.getBoundingClientRect();
var tr=trayEl.getBoundingClientRect();
dbg.textContent=
  'cellPx='+cellPx.toFixed(1)+
  ' | board='+br.width.toFixed(0)+'x'+br.height.toFixed(0)+
  ' | tray='+tr.width.toFixed(0)+'x'+tr.height.toFixed(0)+
  ' | trayCellPx='+trayCellPx().toFixed(1)+
  ' | display-mode='+ (window.matchMedia('(display-mode: standalone)').matches?'standalone':'browser');
}
setInterval(updateDbg,300);
updateDbg();
}
/* ===== HẾT KHỐI DEBUG OVERLAY ===== */

```

Cách tùy biến nội dung hiện ra

Sửa bên trong hàm `updateDbg()` – nối thêm `' | tênBiến='+giá_trị` vào chuỗi `dbg.textContent`. Ví dụ muốn xem thêm kích thước 1 phần tử cụ thể:

```

function updateDbg(){
  var rect =
  document.getElementById('myElement').getBoundingClientRect();
  dbg.textContent =
    'display-mode=' + (...) +
    ' | myElement=' + rect.width.toFixed(0) + 'x' +
    rect.height.toFixed(0);
}

```

Cách xem trên trình duyệt thường (không cần cài PWA)

Cách 4 chỉ tự bật khi `display-mode: standalone` – nếu muốn xem luôn cả lúc test bằng trình duyệt bình thường (chưa cài PWA), đổi điều kiện thành:

```

if(true){ // luôn bật, xem được cả browser lẫn PWA cùng lúc

```

hoặc kết hợp cả 2 origin bằng OR:

```

if(location.search.indexOf('debug=1')!=-1 ||
  window.matchMedia('(display-mode: standalone)').matches){

```

(vào bằng `trang.html?debug=1` trên trình duyệt, hoặc tự động khi mở PWA)

Cách xóa hoàn toàn sau khi debug xong

Bước 1 — Tìm điểm bắt đầu và kết thúc của khối debug trong file, luôn được đánh dấu rõ bằng comment:

```
/* ===... DEBUG OVERLAY — XÓA TOÀN BỘ KHỐI NÀY TRƯỚC KHI DEPLOY ...=== */
...
/* ===...===== HẾT KHỐI DEBUG OVERLAY === */
```

Bước 2 — Xóa nguyên khối từ dòng comment mở đầu đến dòng comment kết thúc (bao gồm cả 2 dòng comment đó).

Bước 3 — Kiểm tra lại: mở file, tìm từ khóa `dbg` hoặc `DEBUG` — nếu không còn kết quả nào là đã xóa sạch.

```
# Cách kiểm tra nhanh bằng dòng lệnh (nếu có quyền truy cập terminal):
grep -n "dbg\|DEBUG OVERLAY" ten-file.html
# Không có kết quả nào in ra = đã xóa sạch
```

Lưu ý khi tái sử dụng cho dự án khác

- Đoạn code này **không phụ thuộc framework** — dùng được cho HTML thuần, React, Vue, bất kỳ gì miễn có JS chạy trong trình duyệt
- `pointer-events:none` trong CSS đảm bảo overlay **không chặn thao tác chạm/click** vào nội dung bên dưới nó — không cần lo che mất nút bấm
- `z-index:99999` đảm bảo luôn nổi lên trên cùng — nếu dự án có modal/popup dùng `z-index` cao hơn, tăng số này lên
- Luôn đặt DÒNG CẢNH BÁO đầu khối debug (như mẫu trên) để lần sau đọc lại code, tự nhắc nhở phải xóa trước khi deploy thật — tránh trường hợp quên và để lộ debug info cho người dùng cuối
- Lưu ý performance:** `setInterval(updateDbg, 300)` chạy liên tục suốt vòng đời trang, kể cả khi không ai nhìn vào overlay — trên máy yếu hoặc nếu quên tắt debug trong lúc để app chạy lâu (test pin, test hiệu năng dài hạn), việc này có thể ảnh hưởng nhẹ đến CPU/pin. Nếu chỉ cần xem giá trị 1 lần chứ không cần cập nhật liên tục, bỏ hẳn dòng `setInterval` và chỉ gọi `updateDbg()` một lần là đủ. Nếu cần cập nhật liên tục nhưng lo về hiệu năng, tăng khoảng cách lên `1000` (1 giây) thay vì `300`.